

# EVIDENCIAS

## CREACION DE USUARIO (POST)

The screenshot shows the Postman interface for a POST request to `http://localhost:3000/api/usuarios`. The request body is a JSON object with the following fields:

```
1 {
2   "nombre": "Nicolás Quintero",
3   "email": "nico@example.com",
4   "password": "123456"
5 }
6
```

The response body is displayed in the bottom panel, showing a success message and the created user details:

```
1 {"mensaje": "Usuario creado correctamente", "usuario": {"id": "7c979605-c2f4-49a8-82cc-e9cdf31b4fae", "nombre": "Nicolás
2 Quintero", "email": "nico@example.com", "password": "$2b$10$Bp6Rmzot1YAMvmLSKMZLX.McOPHmjQTLih40x503apvnbHCm6BS7m",
   "fecha_creacion": "2025-10-05T15:46:11.102Z"}}
3
```

The screenshot shows a web browser window with the address `localhost:3000/api/usuarios/7c979605-c2f4-49a8-82cc-e9cdf31b4fae`. The page displays the JSON response from the API:

```
{
  "id": "7c979605-c2f4-49a8-82cc-e9cdf31b4fae",
  "nombre": "Nicolás Quintero",
  "email": "nico@example.com",
  "password": "$2b$10$Bp6Rmzot1YAMvmLSKMZLX.McOPHmjQTLih40x503apvnbHCm6BS7m",
  "fecha_creacion": "2025-10-05T15:46:11.102Z"
}
```

# LISTAR USUARIOS (GET)

Overview

GET http://localhost:3000

No environment

HTTP

http://localhost:3000/api/usuarios

Save

Share

GET

http://localhost:3000/api/usuarios

Send

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

{

2

3

"nombre": "Nicolás Q. Actualizado"

4

5

}

6

Body

Cookies

Headers (7)

Test Results

200 OK

6 ms

463 B

{}

JSON

Preview

Visualize

1

[

2

{

3

"id": "7c979605-c2f4-49a8-82cc-e9cdf31b4fae",

4

"nombre": "Nicolás Q. Actualizado",

5

"email": "nico@example.com",

6

"password": "\$2b\$10\$Bp6Rmzot1YAMvmLsKMZLX.McOPHmjQTLih40x503apvnbHCm6BS7m",

7

"fecha\_creacion": "2025-10-05T15:46:11.102Z"

8

}

9

]

Online

Find and replace

Console

Runner

Start Proxy

Cookies

Vault

Trash

## ACTUALIZAR USUARIO (PUT)

The screenshot shows the Postman interface for a PUT request. The URL is `http://localhost:3000/api/usuarios/7c979605-c2f4-49a8-82cc-e9cdf31b4fae`. The request body is a JSON object with the following structure:

```
1 {
2   "nombre": "Nicolás Q. Actualizado"
3 }
4
5
6
```

The response status is **200 OK** with a response time of 30 ms and a body size of 506 B. The response body is a JSON object:

```
1 {
2   "mensaje": "Usuario actualizado",
3   "usuario": {
4     "id": "7c979605-c2f4-49a8-82cc-e9cdf31b4fae",
5     "nombre": "Nicolás Q. Actualizado",
6     "email": "nico@example.com",
7     "password": "$2b$10$Bp6Rmzot1YAMvmLsKMZLX.McOPHmjQTLih40x503apvnbHCm6BS7m",
8     "fecha_creacion": "2025-10-05T15:46:11.102Z"
9   }
10 }
```

The screenshot shows a web browser with the address bar displaying `localhost:3000/api/usuarios/7c979605-c2f4-49a8-82cc-e9cdf31b4fae`. Below the browser, there is a code editor with the following JSON content:

```
{
  "id": "7c979605-c2f4-49a8-82cc-e9cdf31b4fae",
  "nombre": "Nicolás Q. Actualizado",
  "email": "nico@example.com",
  "password": "$2b$10$Bp6Rmzot1YAMvmLsKMZLX.McOPHmjQTLih40x503apvnbHCm6BS7m",
  "fecha_creacion": "2025-10-05T15:46:11.102Z"
}
```

At the bottom, there is a checkbox labeled "Dar formato al texto" which is checked.

# ELIMINAR USUARIO (DELETE)

Overview

DEL http://localhost:3000/api/usuarios/7c979605-c2f4-49a8-82cc-e9cdf31b4fae

HTTP

http://localhost:3000/api/usuarios/7c979605-c2f4-49a8-82cc-e9cdf31b4fae

Save

Share

DELETE

http://localhost:3000/api/usuarios/7c979605-c2f4-49a8-82cc-e9cdf31b4fae

Send

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

{

2

"nombre": "Nicolás Q. Actualizado"

3

}

4

5

6

Body

Cookies

Headers (7)

Test Results

200 OK

7 ms

280 B

{}

JSON

Preview

Visualize

1

{

2

"mensaje": "Usuario eliminado correctamente"

3

}

Online

Find and replace

Console

Runner

Start Proxy

Cookies

Vault

Trash

## VERIFICACION DE ELIMININACION (GET)

The screenshot displays the Postman application interface. At the top, the navigation bar includes 'Home', 'Workspaces', and 'API Network'. A search bar and utility buttons like 'Invite', 'Upgrade', and 'Share' are also present. The main workspace shows a GET request to 'http://localhost:3000/api/usuarios'. The 'Body' tab is selected, showing a raw JSON response: 

```
{  "nombre": "Nicolás Q. Actualizado"}
```

. The status bar at the bottom indicates a successful '200 OK' response with a 7 ms duration and 235 B size. The bottom-most bar contains links to 'Online', 'Find and replace', 'Console', 'Runner', 'Start Proxy', 'Cookies', 'Vault', 'Trash', and a help icon.

Home Workspaces **API Network** Search Postman Ctrl K Invite Upgrade

Overview **GET http://localhost:3000/api/usuarios** Save Share

**GET** http://localhost:3000/api/usuarios **Send**

Params Authorization Headers (8) **Body** Scripts Settings Cookies Beautify

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```
1 {
2   "nombre": "Nicolás Q. Actualizado"
3 }
4
5
6
```

**Body** Cookies Headers (7) Test Results 200 OK • 7 ms • 235 B

{ } **JSON** Preview Visualize

1 []

Online Find and replace Console Runner Start Proxy Cookies Vault Trash

## INFORME REFLEXIVO

### Lecciones aprendidas al implementar la API

Durante el desarrollo de esta actividad aprendí el proceso completo de creación de una API REST funcional utilizando Node.js y Express, comprendiendo la estructura básica que debe tener un proyecto backend: rutas, controladores, y manejo de datos.

También entendí cómo los métodos HTTP (GET, POST, PUT y DELETE) permiten la comunicación entre el cliente y el servidor, y la importancia de validar la información que se recibe.

Otra lección importante fue el uso de herramientas como Postman, que permite probar los endpoints de manera práctica, verificando las respuestas del servidor y facilitando la depuración de errores. Además, aprendí el valor de mantener una buena organización del código y el uso de archivos JSON como base de datos temporal para pruebas locales.

### Dificultades encontradas y cómo fueron solucionadas

Al inicio, tuve dificultades con la configuración del entorno de Node.js y la instalación de Yarn, ya que Windows bloqueaba la ejecución de algunos comandos. Esto se solucionó ajustando la política de ejecución en PowerShell y reinstalando correctamente las dependencias.

Otra dificultad fue entender cómo organizar las carpetas y archivos del proyecto (rutas, controladores, datos, etc.), pero mediante la práctica y el repaso de ejemplos, logré comprender la lógica detrás de la arquitectura de Express.

También hubo pequeños errores de sintaxis o rutas mal escritas que fueron resueltos analizando los mensajes de error en la terminal y corrigiendo el código paso a paso.

## Importancia de contar con un backend bien estructurado en aplicaciones móviles

Un backend bien estructurado es esencial para el desarrollo de aplicaciones móviles confiables y escalables, ya que se encarga de procesar los datos, aplicar la lógica de negocio y garantizar la seguridad de la información.

Cuando el backend está bien diseñado, el frontend (la aplicación móvil) puede comunicarse de forma eficiente y segura, sin depender de procesos desordenados o datos inconsistentes.

Además, un backend organizado facilita el mantenimiento del sistema, permite agregar nuevas funcionalidades sin romper las existentes y hace posible que varios desarrolladores trabajen en el mismo proyecto sin generar conflictos.

En conclusión, un backend sólido es la base que asegura el correcto funcionamiento de cualquier aplicación moderna.